

Machine Learning Algorithms Laboratory

Experiment 2 Report

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naïve Bayes and KNN 1
Student Name	Gautham S
Register Number	3122247001016
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

The objective of this experiment is to build spam classifiers using Naïve Bayes and KNN algorithms. The Gaussian, Multinomial and Bernoulli variants of Naïve Bayes are compared. The effect of varying k in KNN is studied and KDTree and BallTree are compared. GridSearchCV and RandomizedSearchCV are used to find the best KNN hyperparameters. The models are also evaluated using K-Fold Cross Validation and their computational time is compared.

2 Problem Statement

Given the Spambase dataset, the task is to classify each email as spam or ham (not spam) based on the frequency of certain words and characters present in the email. Different classifiers are trained on this data and their performance is compared using standard evaluation metrics.

3 Dataset Description

Dataset	Description
Dataset Name	Spambase Dataset
Dataset Source	Kaggle https://www.kaggle.com/datasets/somesh24/spambase
Number of Samples	4601 (391 duplicate rows removed, 4210 used)
Number of Features	57 numerical features (word frequency, character frequency and capital run length features)
Number of Classes	2 (0 = Ham, 1 = Spam)
Missing Values	No missing values
Train-Test Split	80% Train (3368 samples), 20% Test (842 samples)

4 Exploratory Data Analysis

4.1 Class Distribution

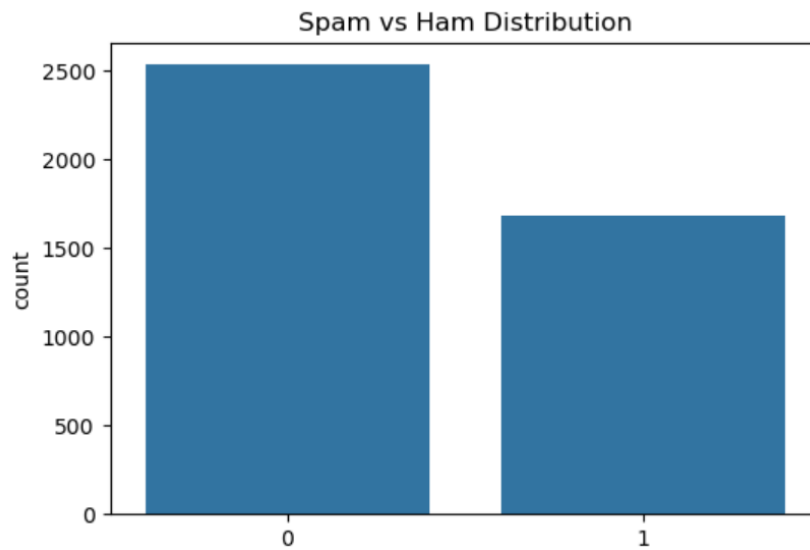


Figure 1: Distribution of Spam and Ham Emails

Inference: The dataset has more ham emails than spam emails. There are 2531 ham samples and 1679 spam samples after removing duplicates. Since the classes are not extremely imbalanced, accuracy can still be used as a metric, but precision, recall and F1-score are also checked to be sure.

4.2 Correlation Heatmap

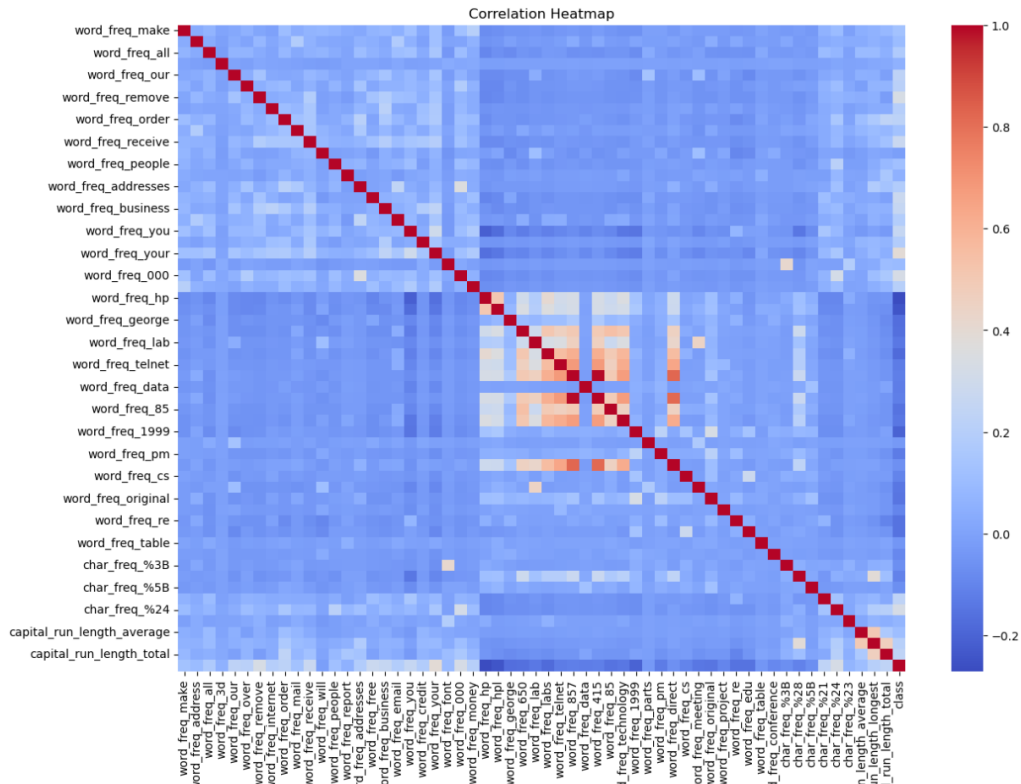


Figure 2: Correlation Heatmap of Spambase Features

Inference: Most of the word frequency features are not strongly correlated with each other. A few features like the capital run length features show some correlation among themselves. This heatmap helps in getting a rough idea about redundant features, although no features were removed in this experiment.

4.3 Important Features

The mean value of each feature was compared between the two classes to find the features that separate spam and ham the most. The top 5 features found were `capital_run_length_total`, `capital_run_length_longest`, `capital_run_length_average`, `word_freq_your` and `word_freq_you`.

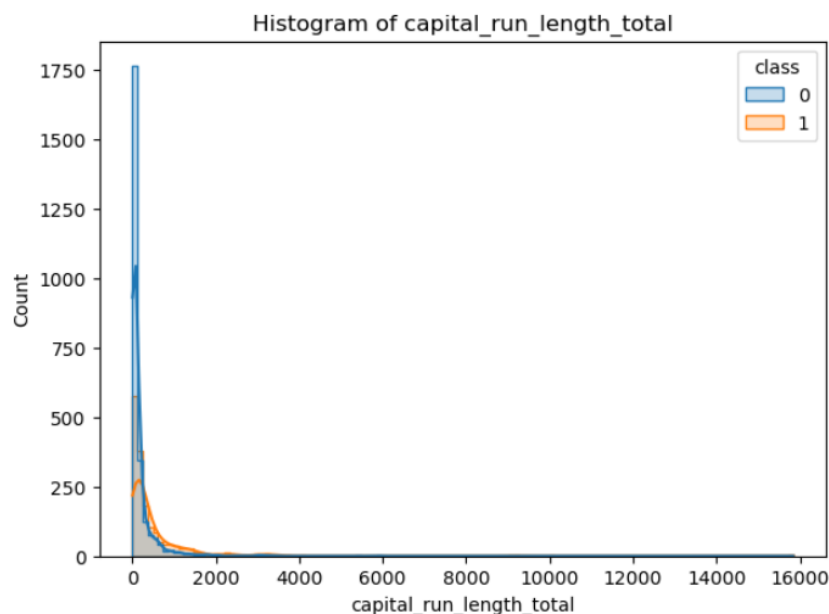


Figure 3: Histogram of capital_run_length_total by Class

Inference: Spam emails tend to have a much higher total capital run length compared to ham emails. This makes sense because spam mails often use a lot of capital letters to catch attention. The word "your" showed a similar trend, appearing more often in spam mails since they usually try to address the reader directly (like "your account", "your prize").

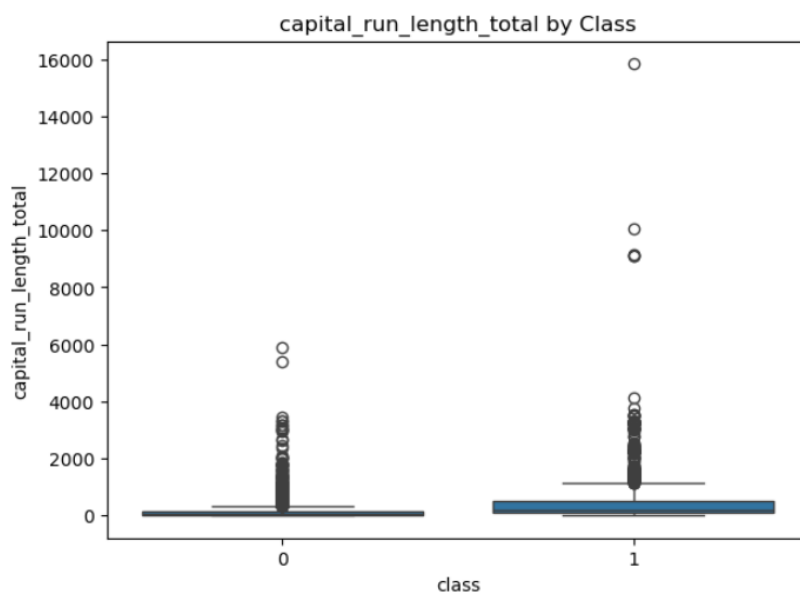


Figure 4: Boxplot of capital_run_length_total by Class

Inference: The boxplot confirms the histogram observation. Spam emails have a higher median and more spread in the capital run length feature. There are also a few extreme outliers present in both classes.

5 Data Preprocessing

The dataset had 391 duplicate rows which were removed before training. There were no missing values in the dataset. All the features are numerical (frequency based) so no encoding was required. The data was split into 80% training and 20% testing using stratified sampling so that the class ratio remains same in both sets.

Since KNN and Gaussian NB are distance/probability based, the features were standardized using `StandardScaler`. For Multinomial NB the original (non-negative) feature values were used since it does not work with negative or scaled values. For Bernoulli NB the features were converted into binary values (whether a word/character is present or not) since Bernoulli NB expects binary features.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# bernoulli nb needs binary features
X_train_binary = (X_train > 0).astype(int)
X_test_binary = (X_test > 0).astype(int)
```

6 Machine Learning Algorithms

6.1 Naïve Bayes

Naïve Bayes is a probabilistic classifier based on Bayes' Theorem, assuming that the features are conditionally independent given the class.

$$P(C|X) = \frac{P(X|C) P(C)}{P(X)} \quad (1)$$

- **Gaussian NB** assumes that each feature follows a normal (Gaussian) distribution.
- **Multinomial NB** is generally used for count/frequency type data.
- **Bernoulli NB** is used when features are binary, i.e. present or absent.

6.2 K-Nearest Neighbours (KNN)

KNN is a lazy learning, instance based algorithm. It does not really "train" a model, instead it stores the training data and at prediction time it finds the k nearest points to the query point (usually using Euclidean distance) and assigns the majority class among them.

KDTree and BallTree are data structures used to speed up the neighbour search instead of comparing the query point with every training point (brute force).

7 Experimental Setup

Python Version	3.11
Scikit-Learn Version	1.4
IDE	Jupyter Notebook
Operating System	Windows 11
Hardware	Intel Core i5, 16GB RAM

8 Model Training and Evaluation

A small helper function was written to train a model and note down the time taken, so the same function could be reused for all the classifiers instead of repeating the code again and again.

```
def evaluate_model(model, X_train, X_test, y_train, y_test):
    t1 = time.perf_counter()
    model.fit(X_train, y_train)
    t2 = time.perf_counter()

    y_pred = model.predict(X_test)
    t3 = time.perf_counter()

    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    # ... other metrics calculated similarly

    return acc, f1, (t2-t1), (t3-t2)
```

8.1 Naïve Bayes Comparison

Metric	Gaussian	Multinomial	Bernoulli
Accuracy	0.8290	0.7886	0.8943
Precision	0.7143	0.7616	0.8777
Recall	0.9524	0.6845	0.8542
F1	0.8163	0.7210	0.8658
ROC-AUC	0.9471	0.8342	0.9609

Table 2: Comparison of Naïve Bayes Variants

Inference: Bernoulli NB gave the best accuracy and F1-score among the three variants. Gaussian NB has the highest recall, meaning it catches most of the spam emails but also gives more false positives (lower precision). Multinomial NB performed the worst here, probably because the word frequency features are continuous and not exactly count data.

8.2 KNN for Different Values of k

k	Accuracy	Precision	Recall	F1
1	0.9062	0.8836	0.8810	0.8823
3	0.9097	0.9063	0.8631	0.8841
5	0.9133	0.9148	0.8631	0.8882
7	0.9050	0.9103	0.8452	0.8765
9	0.9074	0.9243	0.8363	0.8781
11	0.9086	0.9164	0.8482	0.8810

Table 3: KNN Performance for Different k values

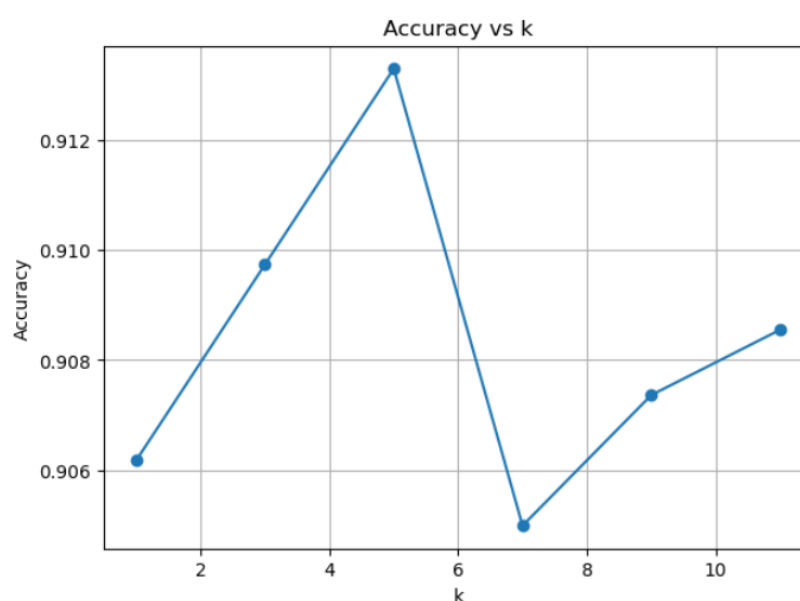


Figure 5: Accuracy vs k

Inference: The accuracy is highest around $k=5$ and drops slightly for very small ($k=1$, overfitting) and larger values of k . This shows that $k=5$ gives a good balance between bias and variance for this dataset. For very small k the model is too sensitive to noise, and for very large k it starts underfitting.

8.3 GridSearchCV vs RandomizedSearchCV

Parameter	GridSearchCV	RandomizedSearchCV
Best k	9	9
Weights	distance	distance
Algorithm	auto	ball_tree
CV Accuracy	0.9029	0.9029
Execution Time (s)	3.3535	0.7691

Table 4: GridSearchCV vs RandomizedSearchCV

```

param_grid = {
    "n_neighbors": [1, 3, 5, 7, 9, 11],
    "weights": ["uniform", "distance"],
    "algorithm": ["auto", "ball_tree", "kd_tree"]
}

grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5,
                    scoring="accuracy")
grid.fit(X_train_scaled, y_train)

print(grid.best_params_)
print(grid.best_score_)

```

Inference: Both GridSearchCV and RandomizedSearchCV found the same best k (9) and weights (distance) with the same CV accuracy of 0.9029. However RandomizedSearchCV took much less time (0.77s vs 3.35s) since it only tries a fixed number of random combinations instead of every possible combination.

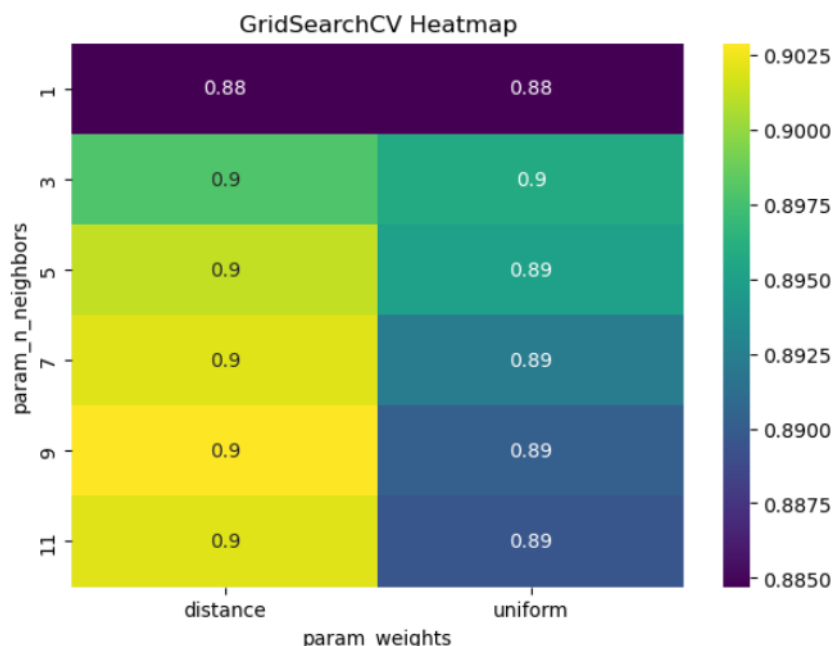


Figure 6: GridSearchCV Heatmap (Mean CV Accuracy)

Inference: The heatmap shows that "distance" weighting generally gives slightly better accuracy than "uniform" weighting across most values of k, and accuracy peaks around k=9.

Most of the randomly sampled parameter combinations in RandomizedSearchCV achieved a similar CV accuracy in a close range, which means the model performance is not too sensitive to the exact combination as long as k is in a reasonable range.

8.4 KDTree vs BallTree

Metric	KDTree	BallTree
Accuracy	0.9228	0.9228
Training Time (s)	0.0146	0.0082
Prediction Time (s)	0.0993	0.0946

Table 5: KDTree vs BallTree Comparison

Inference: Both KDTree and BallTree give the exact same accuracy since they are just different ways of organizing the data for faster neighbour search, they don't change the result. BallTree was slightly faster in this case for both training and prediction, but the difference is very small for this dataset size.

8.5 5-Fold Cross Validation

Fold	Naïve Bayes	Best KNN
1	0.8101	0.9050
2	0.8101	0.9006
3	0.8101	0.9036
4	0.8351	0.8975
5	0.8336	0.9079
Average	0.8198	0.9029

Table 6: 5-Fold Cross Validation Results

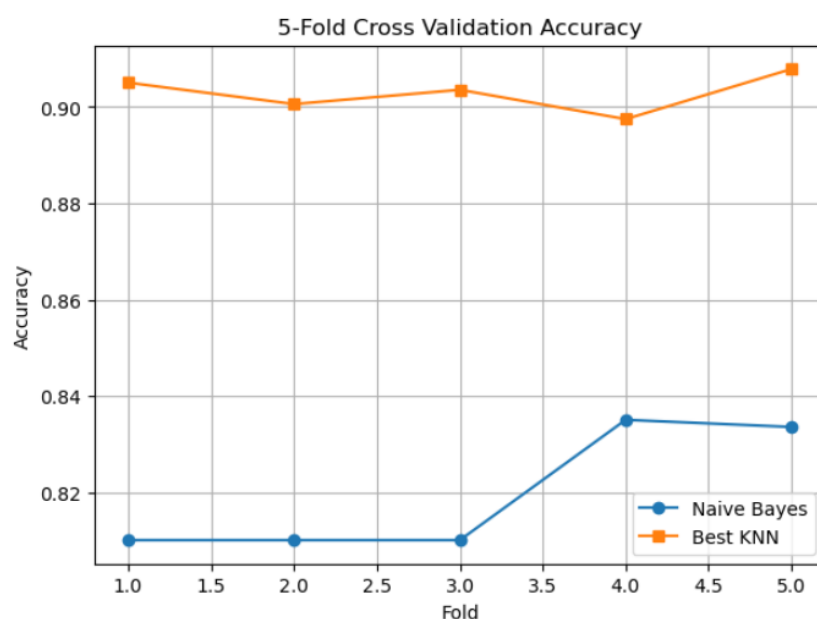


Figure 7: 5-Fold Cross Validation Accuracy

Inference: The best KNN model is consistently better than Gaussian Naïve Bayes across all 5 folds. The accuracy of KNN is also more stable (less variation) between folds compared to Naïve Bayes.

9 Time Complexity Analysis

9.1 Theoretical Time Complexity

Algorithm	Training	Prediction
Naïve Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)$ avg
BallTree	$O(n \log n)$	$O(\log n)$ avg

Table 7: Theoretical Time Complexity (n = samples, d = features)

9.2 Experimental Time Analysis

Algorithm	Training(s)	Prediction(s)
Gaussian NB	0.002255	0.000753
Multinomial NB	0.001419	0.000555
Bernoulli NB	0.002436	0.000842
Best KNN	0.002184	0.006734

Table 8: Experimental Training and Prediction Time

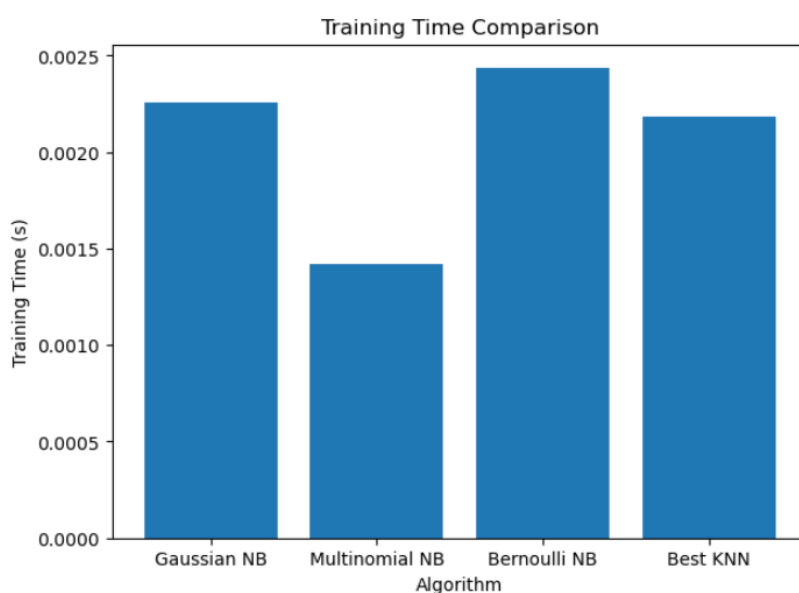


Figure 8: Training Time Comparison

Inference: All the Naïve Bayes variants train almost instantly since they just calculate simple statistics (mean, variance or probabilities) from the data. KNN also has a low training time because it just stores the data. But at prediction time KNN is noticeably slower than all the Naïve Bayes variants because it needs to calculate distances to find the nearest neighbours, which matches the theoretical complexity.

10 Result Visualization

10.1 Confusion Matrices

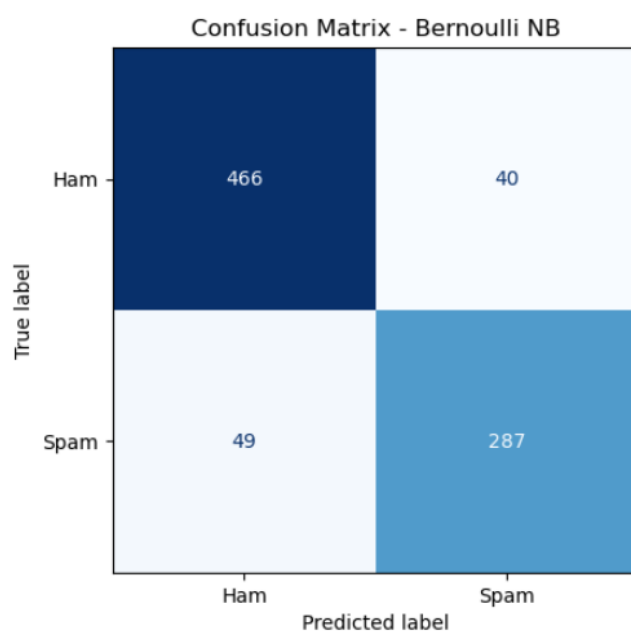


Figure 9: Confusion Matrix - Bernoulli NB

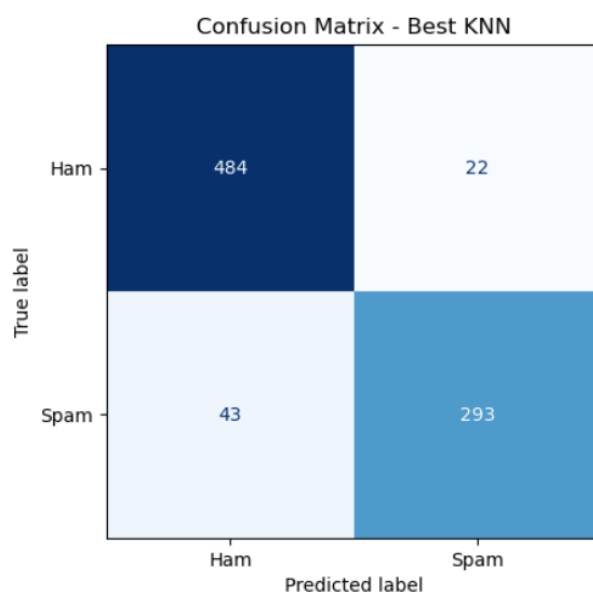


Figure 10: Confusion Matrix - Best KNN

Inference: Bernoulli NB and KNN both have a fairly balanced confusion matrix with relatively few misclassifications. Gaussian NB (see accuracy/precision table above) had noticeably more false positives, which is why its precision was lower even though its recall was the highest.

10.2 ROC Curve

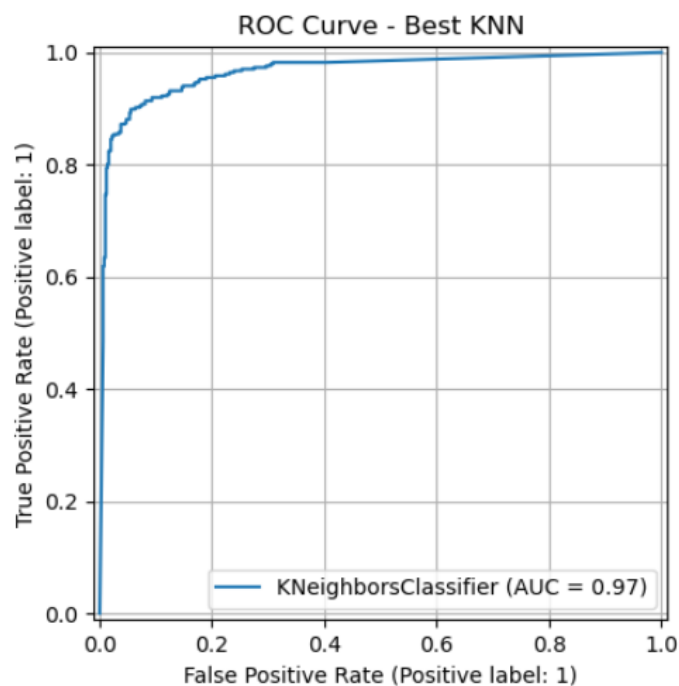


Figure 11: ROC Curve - Best KNN

Inference: The ROC curve for the best KNN model stays close to the top-left corner, giving a high AUC. From the Naïve Bayes table, Bernoulli NB had the highest ROC-AUC (0.96) followed by Gaussian NB (0.95), while Multinomial NB was the lowest (0.83) among the three variants.

10.3 Overall Classifier Comparison

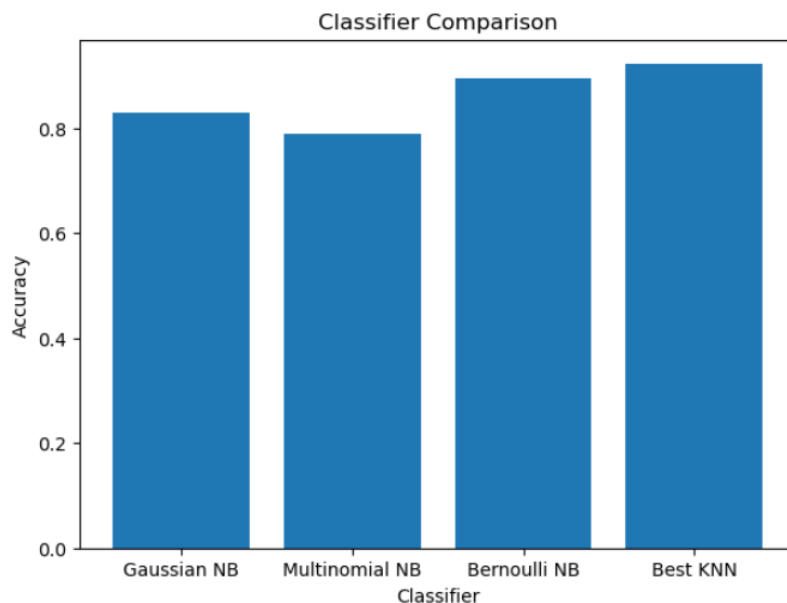


Figure 12: Accuracy Comparison of All Classifiers

Inference: Among all the models tested, Bernoulli NB and the tuned KNN model gave the highest accuracy, both performing better than Gaussian NB and Multinomial NB.

11 Discussion

This experiment helped in understanding how Naïve Bayes and KNN work differently even though both are relatively simple algorithms. Naïve Bayes variants train almost instantly since they only estimate probability distributions from the data, while KNN has almost no training cost but a higher prediction cost since it has to search for neighbours every time. Hyperparameter tuning using GridSearchCV and RandomizedSearchCV helped in finding a better KNN configuration than the default. Overall, Bernoulli NB and the tuned KNN model were the best performing classifiers for this spam classification task.

12 Conclusion

In this experiment, spam emails were classified using Gaussian, Multinomial and Bernoulli Naïve Bayes and also using KNN. Bernoulli NB gave the best performance among the Naïve Bayes variants, and KNN with $k=9$ and distance weighting (found using GridSearchCV/RandomizedSearchCV) gave comparable results. The experimental time analysis matched the theoretical time complexity, with Naïve Bayes being faster at prediction than KNN. This experiment gave a good understanding of probabilistic and instance-based classifiers along with hyperparameter tuning techniques.

13 References

1. Scikit-learn Documentation. <https://scikit-learn.org/stable/>
2. Spambase Dataset, Kaggle. <https://www.kaggle.com/datasets/somesh24/spambase>
3. Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow.
4. Pattern Recognition and Machine Learning.